LAPORAN PRAKTIKUM

Struktur Data

MODUL 4

SINGLY LINKED LIST (BAGIAN PERTAMA)



Disusun oleh:

Farel Tri Julian

109082500163

S1IF-13-02

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

TAHUN 2026

LATIHAN SOAL MODUL

Link Repo Github Modul 4 : https://github.com/FarelTriJulian/Struktur-Data/tree/main/modul4

1. **Soal 1**

Source Code

```
#include <iostream>
#include <string>
using namespace std;

struct Node { string nama; Node* next; };
Node* head = nullptr;

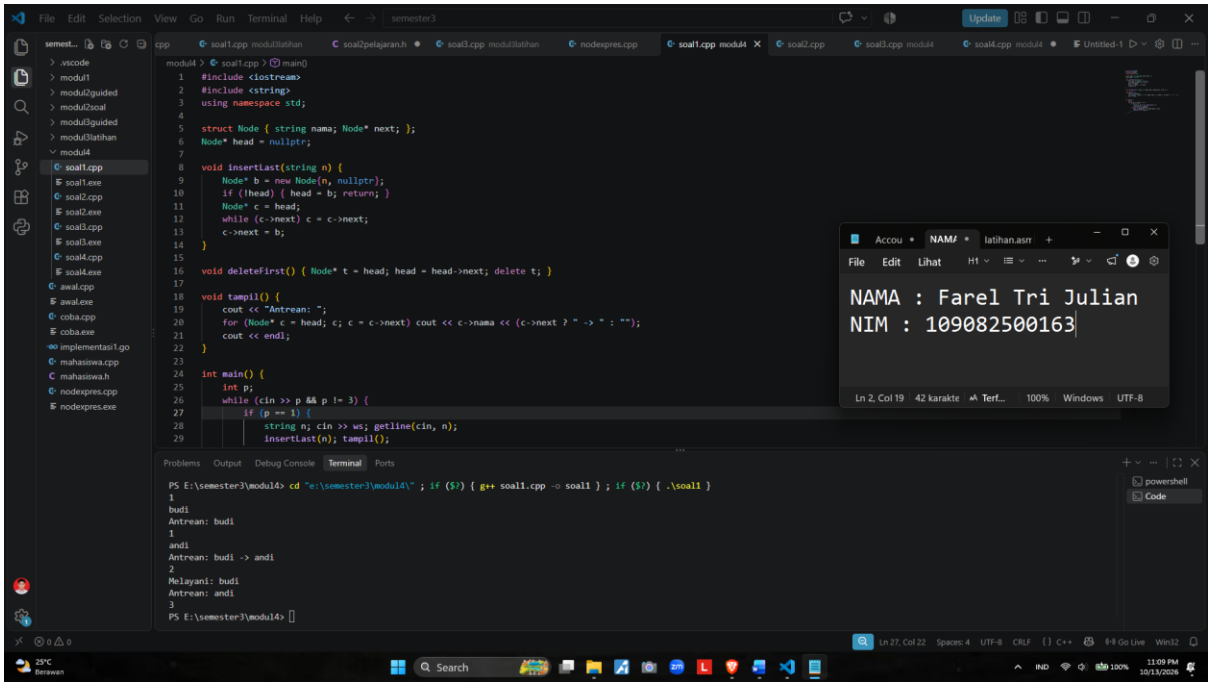
void insertLast(string n) {
    Node* b = new Node{n, nullptr};
    if (!head) { head = b; return; }
    Node* c = head;
    while (c->next) c = c->next;
    c->next = b;
}

void deleteFirst() { Node* t = head; head = head->next;
delete t; }

void tampil() {
    cout << "Antrean: ";
    for (Node* c = head; c; c = c->next) cout << c->nama <<
(c->next ? " -> " : "");
    cout << endl;
}

int main() {
    int p;
    while (cin >> p && p != 3) {
        if (p == 1) {
            string n; cin >> ws; getline(cin, n);
            insertLast(n); tampil();
        } else if (p == 2 && head) {
            cout << "Melayani: " << head->nama << endl;
            deleteFirst(); tampil();
        }
    }
}
```

Screenshoot program



Deskripsi program

Program tersebut merupakan antrean nasabah bank menggunakan Singly Linked List, di mana setiap node menyimpan nama nasabah dan pointer ke node berikutnya. Fungsi insertLast menambahkan nasabah baru di akhir antrean, deleteFirst menghapus nasabah di urutan pertama, dan tampil mencetak seluruh isi antrean dengan pemisah “->”. Di main, program membaca pilihan menu secara berulang: pilihan 1 meminta nama nasabah lalu menambahkannya ke antrean, pilihan 2 mencetak nama nasabah terdepan sebagai yang sedang dilayani lalu menghapusnya dari antrean, dan pilihan 3 menghentikan program.

2. Soal 2

Source Code

```
#include <iostream>
using namespace std;

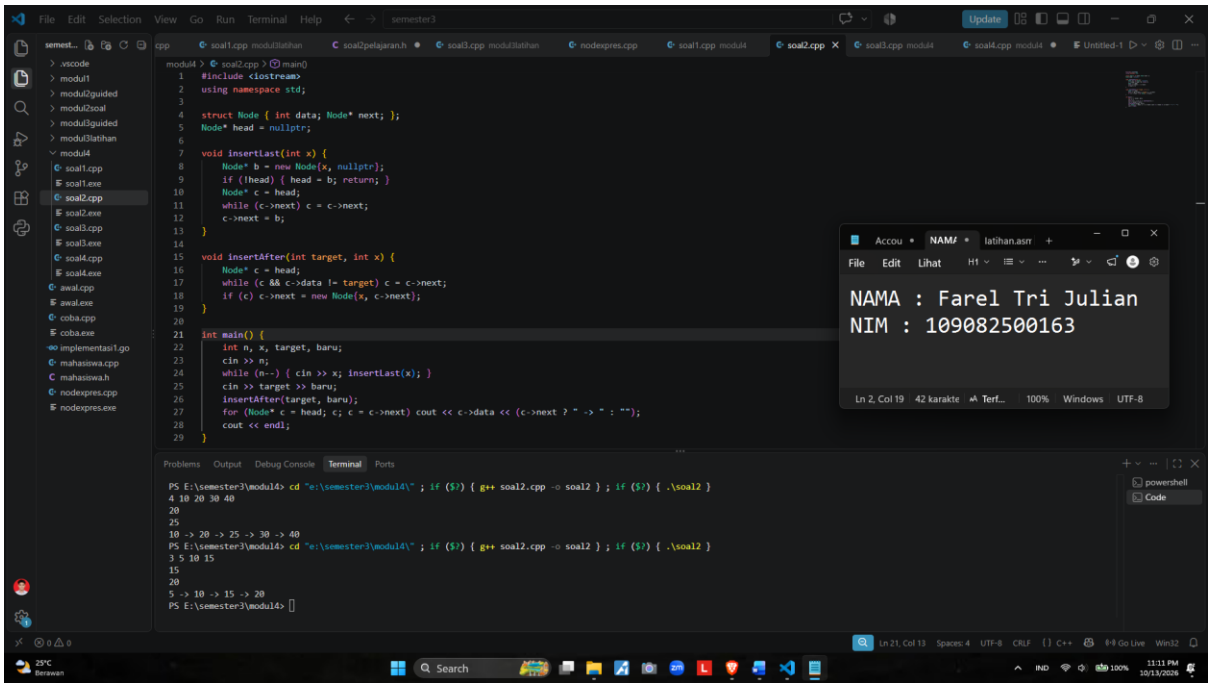
struct Node { int data; Node* next; };
Node* head = nullptr;

void insertLast(int x) {
    Node* b = new Node{x, nullptr};
    if (!head) { head = b; return; }
    Node* c = head;
    while (c->next) c = c->next;
    c->next = b;
}
```

```
void insertAfter(int target, int x) {
    Node* c = head;
    while (c && c->data != target) c = c->next;
    if (c) c->next = new Node{x, c->next};
}

int main() {
    int n, x, target, baru;
    cin >> n;
    while (n--) { cin >> x; insertLast(x); }
    cin >> target >> baru;
    insertAfter(target, baru);
    for (Node* c = head; c; c = c->next) cout << c->data <<
(c->next ? " -> " : "");
    cout << endl;
}
```

Screenshoot program



Deskripsi program

Program tersebut merupakan Singly Linked List yang menyimpan bilangan bulat, di mana setiap node berisi data dan pointer next ke node berikutnya. Fungsi insertLast menambahkan angka di akhir list, sedangkan insertAfter mencari node yang berisi nilai target lalu menyisipkan node baru tepat setelahnya tanpa memutus sambungan ke sisa list (jika target tidak ditemukan, list tidak berubah). Di main, program membaca N lalu N angka untuk membentuk list awal, kemudian membaca angka yang dicari dan angka baru

yang akan disisipkan, memanggil insertAfter, dan terakhir mencetak seluruh isi list dengan pemisah ->.

3. **Soal 3**

Source Code

```
#include <iostream>
using namespace std;

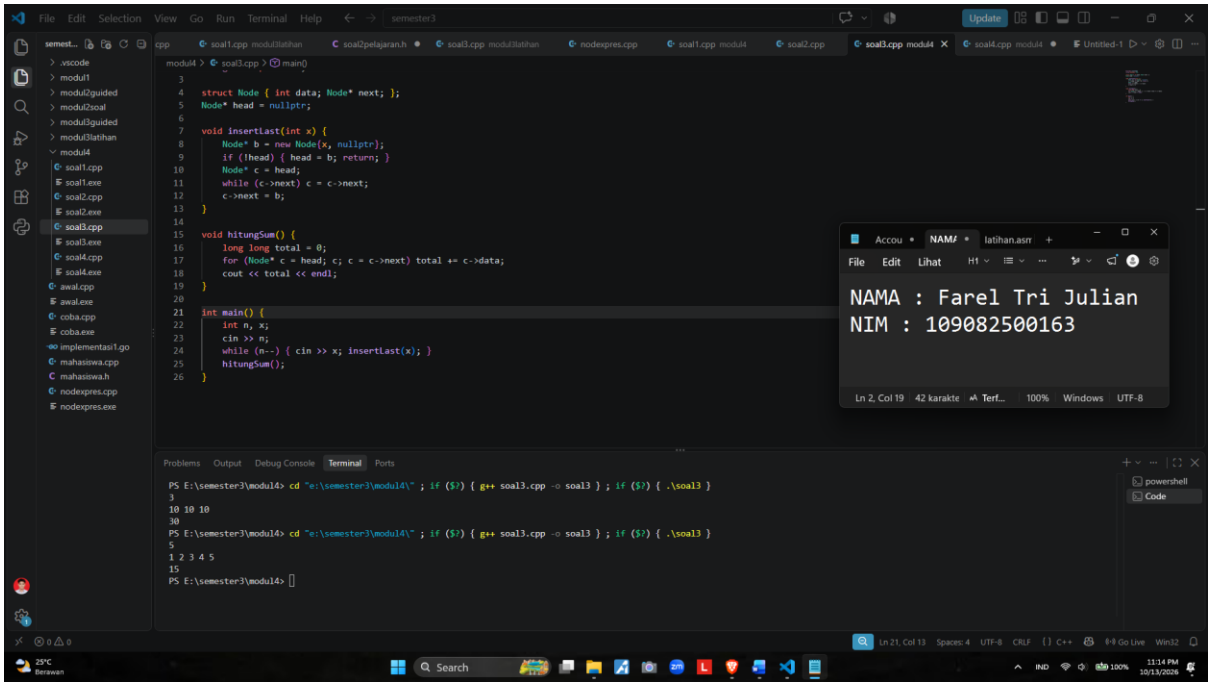
struct Node { int data; Node* next; };
Node* head = nullptr;

void insertLast(int x) {
    Node* b = new Node{x, nullptr};
    if (!head) { head = b; return; }
    Node* c = head;
    while (c->next) c = c->next;
    c->next = b;
}

void hitungSum() {
    long long total = 0;
    for (Node* c = head; c; c = c->next) total += c->data;
    cout << total << endl;
}

int main() {
    int n, x;
    cin >> n;
    while (n--) { cin >> x; insertLast(x); }
    hitungSum();
}
```

Screenshoot program



Deskripsi program

Program tersebut merupakan Singly Linked List yang menyimpan bilangan bulat untuk menghitung jumlah seluruh elemennya. Fungsi insertLast menambahkan angka di akhir list, sedangkan prosedur hitungSum menelusuri list dari head sampai node terakhir sambil menjumlahkan data setiap node ke variabel total, lalu mencetak hasilnya. Di main, program membaca N lalu N angka untuk dimasukkan ke list, kemudian memanggil hitungSum() untuk menampilkan total penjumlahan.

4. Soal 4

Source Code

```
#include <iostream>
#include <string>
using namespace std;

struct Buku { string judul; Buku* next; };
Buku* head = nullptr;

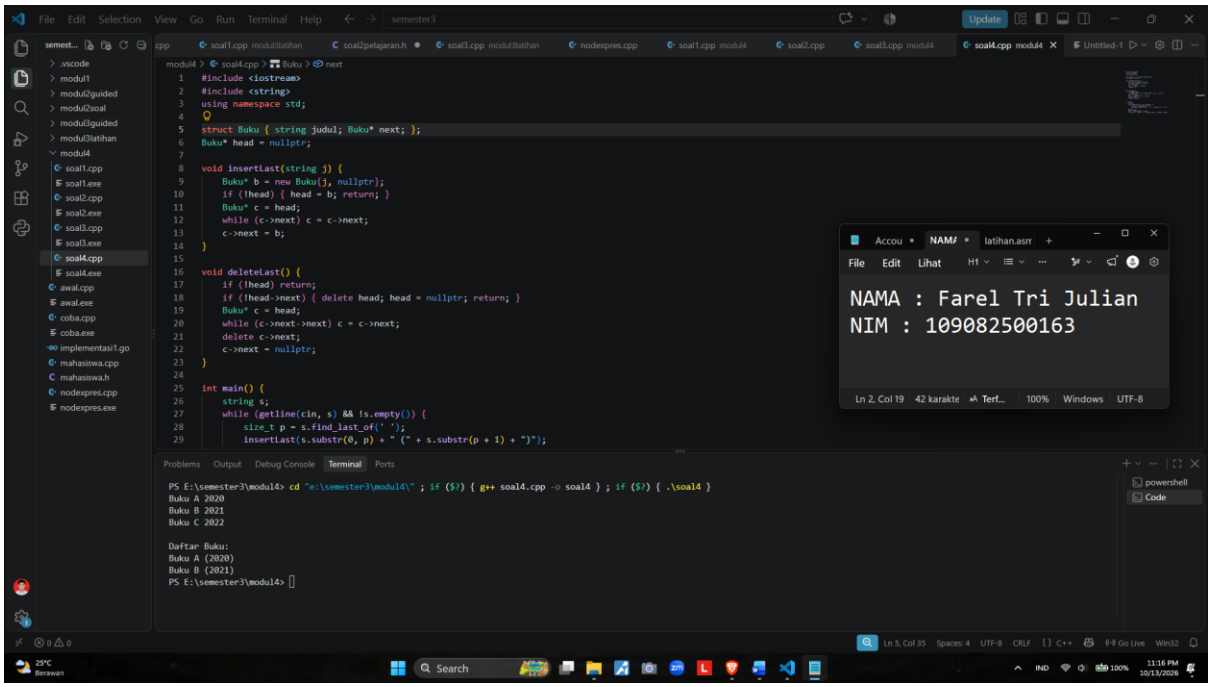
void insertLast(string j) {
    Buku* b = new Buku{j, nullptr};
    if (!head) { head = b; return; }
    Buku* c = head;
    while (c->next) c = c->next;
    c->next = b;
}

void deleteLast() {
```

```
        if (!head) return;
        if (!head->next) { delete head; head = nullptr; return; }
        Buku* c = head;
        while (c->next->next) c = c->next;
        delete c->next;
        c->next = nullptr;
    }

    int main() {
        string s;
        while (getline(cin, s) && !s.empty()) {
            size_t p = s.find_last_of(' ');
            insertLast(s.substr(0, p) + " (" + s.substr(p + 1) +
    ")");
        }
        deleteLast();
        cout << "Daftar Buku:" << endl;
        for (Buku* c = head; c; c = c->next) cout << c->judul <<
    endl;
    }
```

Screenshoot program



Deskripsi program

Program tersebut mengimplementasikan Singly Linked List untuk mengelola data buku perpustakaan, di mana setiap node menyimpan satu string judul yang sudah berisi judul beserta tahun terbit, dan pointer next ke node berikutnya. Fungsi insertLast menambahkan buku di akhir list, sedangkan deleteLast menghapus buku di urutan terakhir dengan menangani tiga kondisi: list kosong, list yang hanya berisi satu node, dan

list dengan banyak node. Di main, program membaca data buku per baris sampai menemukan baris kosong, memisahkan kata terakhir sebagai tahun dan sisanya sebagai judul, lalu menyimpannya dalam format Judul. Setelah itu deleteLast() dipanggil satu kali dan daftar buku yang tersisa dicetak satu per baris di bawah tulisan “Daftar Buku:”.

5. **Soal 5 (guided)**

Source Code

```
#include <iostream>

using namespace std;

typedef int infotype;
typedef struct elmlist *address;

struct elmlist {
    infotype info;
    address next;
};

struct List {
    address first;
};

void createList(List &L) {
    L.first = NULL;
}

address alokasi(infotype x) {
    address P = new elmlist;
    P->info = x;
    P->next = NULL;
    return P;
}

void dealokasi(address &P) {
    delete P;
    P = NULL;
}

void insertFirst(List &L, address P) {
    P->next = L.first;
    L.first = P;
}

void insertLast(List &L, address P) {
    if (L.first == NULL) {
        insertFirst(L, P);
    } else {
```

```
            address Q = L.first;
            while (Q->next != NULL) {
                Q = Q->next;
            }
            Q->next = P;
        }
    }

    void insertAfter(List &L, address P, address Prec) {
        if (Prec != NULL) {
            P->next = Prec->next;
            Prec->next = P;
        }
    }

    void deleteFirst(List &L, address &P) {
        if (L.first != NULL) {
            P = L.first;
            L.first = P->next;
            P->next = NULL;
        }
    }

    void deleteLast(List &L, address &P) {
        if (L.first != NULL) {
            if (L.first->next == NULL) {
                deleteFirst(L, P);
            } else {
                address Q = L.first;
                while (Q->next->next != NULL) {
                    Q = Q->next;
                }
                P = Q->next;
                Q->next = NULL;
            }
        }
    }

    void deleteAfter(List &L, address &P, address Prec) {
        if (Prec != NULL && Prec->next != NULL) {
            P = Prec->next;
            Prec->next = P->next;
            P->next = NULL;
        }
    }

    void printInfo(List L) {
        address P = L.first;
```

```
        while (P != NULL) {
            cout << P->info << ' ';
            P = P->next;
        }
        cout << '\n';
    }

    int main() {
        List L;
        address P1, P2, P3, P4, P5;

        createList(L);

        P1 = alokasi(2);
        insertFirst(L, P1);

        P2 = alokasi(0);
        insertFirst(L, P2);

        P3 = alokasi(8);
        insertFirst(L, P3);

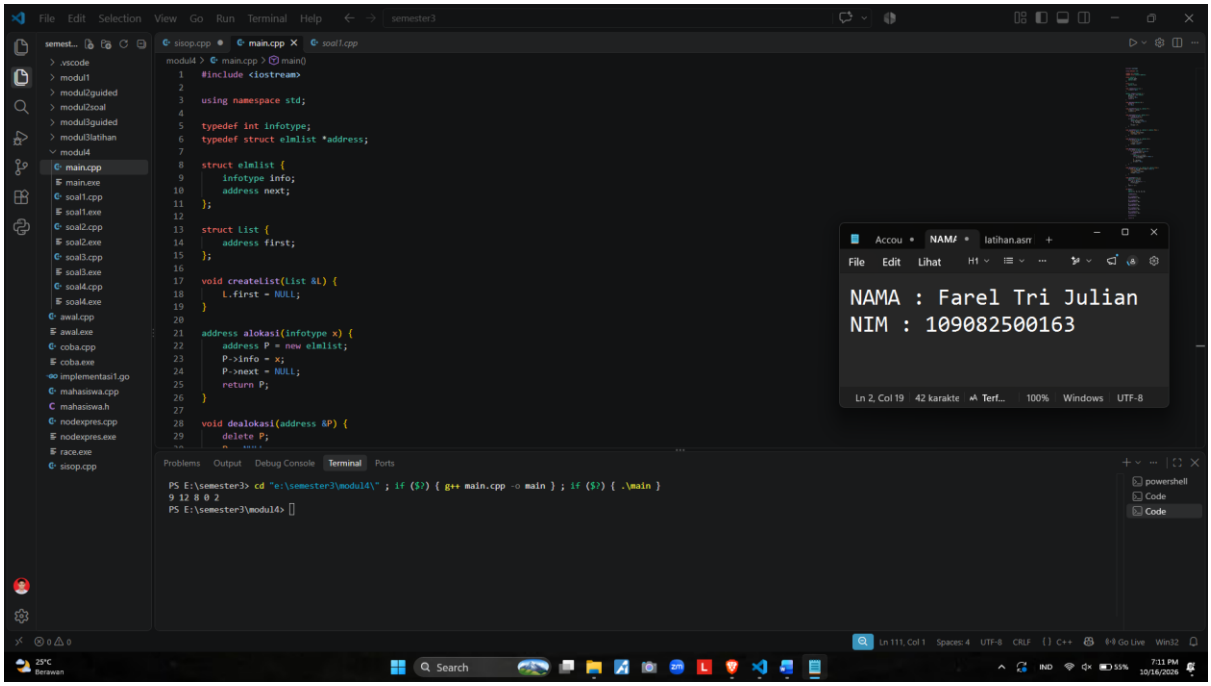
        P4 = alokasi(12);
        insertFirst(L, P4);

        P5 = alokasi(9);
        insertFirst(L, P5);

        printInfo(L);

        return 0;
    }
```

Screenshoot program



Deskripsi program

Program tersebut merupakan implementasi struktur data Singly Linked List (senarai berantai tunggal) dalam bahasa C++ yang mendefinisikan operasi-operasi dasar seperti pembuatan list, alokasi/dealokasi memori, penambahan elemen, penghapusan elemen, serta pencetakan isi list. Pada fungsi main, program mendemonstrasikan penggunaan list ini dengan membuat lima buah node berisi nilai integer (2, 0, 8, 12, dan 9) yang dimasukkan satu per satu ke posisi paling depan menggunakan fungsi insertFirst, sehingga ketika list tersebut dicetak, angka-angka tersebut akan ditampilkan dalam urutan terbalik dari urutan penyisipannya, yaitu 9 12 8 0 2.